

Designing Mobile Rendering Engines with "Bindless" Vulkan

Sergey Kosarevsky
Meta Platforms Inc
Menlo Park, CA, USA
linderdaum@gmail.com

Alexey Medvedev
Meta Platforms Inc
Menlo Park, CA, USA
rudybear@gmail.com



Figure 1: LightweightVK[Kosarevsky 2024] cross-platform bindless Vulkan 1.3 demo.

KEYWORDS

Vulkan 1.3, bindless, mobile

ACM Reference Format:

Sergey Kosarevsky and Alexey Medvedev. 2024. Designing Mobile Rendering Engines with "Bindless" Vulkan. In *Special Interest Group on Computer Graphics and Interactive Techniques Conference Talks (SIGGRAPH Talks '24)*, July 27–August 01, 2024, Denver, CO, USA. ACM, New York, NY, USA, 2 pages. <https://doi.org/10.1145/3641233.3664326>

1 ABSTRACT

In this talk, we'll discuss our experience with designing mobile rendering engines using Vulkan 1.3 "bindless" techniques. We'll show how we've adapted existing "bindful" code to harness these

"bindless" methods on Meta's Quest Pro and Quest 3 series of AR/VR devices and other mobile platforms. Our talk is aimed at an audience already familiar with Vulkan and interested in delving into its "bindless" features.

2 INTRODUCTION

Bindless rendering opens up possibilities for completely new 3D engine designs with GPU-driven rendering and raytracing in mind. Challenges arise in transitioning existing code from older APIs, requiring developers to rethink previous approaches. Modern GPU-driven rendering benefits from unrestricted access to GPU resources, textures and buffers, operating in a "bindless" manner. The CPU role is primarily reduced to managing the active set of resources in GPU memory. Modern mobile GPUs have the necessary hardware features to support this approach[Lista 2024].

Many existing mobile rendering engines adhere to the "bindful" design paradigm. This old approach requires the binding of new sets of resources prior to each draw call, frequently through the complex allocation and updating of Vulkan descriptor sets or, in some cases, resorting to emulating outdated slot-based binding APIs as in OpenGL. This process is seen as inevitable due to limitations

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).
SIGGRAPH Talks '24, July 27–August 01, 2024, Denver, CO, USA
© 2024 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-0515-1/24/07
<https://doi.org/10.1145/3641233.3664326>

in older hardware, particularly on mobile platforms. Consequently, developers write complex code sacrificing clarity and performance.

Modern mobile GPUs now support Vulkan 1.3 and can implement this "bindless" scheme. The main blocker is not missing GPU features but performance. Specifically, performance of indirect memory loads. The situation started to change with the release of ARM's Immortalis GPU, which introduced hardware ray tracing on mobile [ARM 2023]. Ray tracing implies access to resources in a "bindless" way, so it was significantly optimized, and the "bindless" approach is becoming feasible.

In this talk, we'll discuss our experience with designing mobile Vulkan 1.3 "bindless" renderers. Moreover, we'll show how we adapted existing "bindful" code to harness these "bindless" methods on the Quest Pro and Quest 3 series of AR/VR devices and other mobile platforms. We'll also explore the pros and cons of adopting Vulkan 1.3 as the sole rendering API across multiple platforms, including macOS [Kuznetsov 2024] and Android.

3 DESIGN AND IMPLEMENTATION

Let's talk about design decisions involved in transforming "bindful" IGL [Kosarevsky and Medvedev 2024] into a "bindless" Vulkan 1.3 wrapper, LightweightVK [Kosarevsky 2024]. To do this, we utilize descriptor indexing and buffer device address.

LightweightVK manages sparse collections of resources like images, image views, samplers, and buffers. Image views and samplers are packed into large Vulkan descriptor sets, which are partially bound. With the descriptorIndexing feature, each image view and sampler can be accessed individually by its index. Buffers are accessed directly via bufferDeviceAddress, with GPU addresses passed into shaders as uint64_t values using Vulkan push-constants. As resources are loaded and unloaded, Vulkan descriptor sets are updated using update-after-bind functionality.

LightweightVK condenses interfaces responsible for textures and buffers into a handful of functions. GPU addresses are ready for immediate use, while access to textures is provided via handles¹. Contrary to Meta's IGL library, LightweightVK does not contain any binding functions except for vertex/index buffers and pipelines. To manage Vulkan resources, it creates a single descriptor set with three binding points: one for textures, one for samplers, and one for storage images. The maximum number of descriptors in the set is based on the number of currently loaded textures. When more textures are loaded, a new larger descriptor set is created.

All resources are represented as uint32_t indices or uint64_t GPU addresses. These integers can be stored directly in per-material data. Material data can be packed into buffers, with each material holding the necessary texture and buffer ids for rendering. To supply data to shaders, we can use a technique detailed in "3D Graphics Rendering Cookbook" [Kosarevsky and Latypov 2021]. We utilize the GLSL built-in variable gl_BaseInstance. It holds a value from VkDrawIndexedIndirectCommand::firstInstance for the current draw call and can contain any user-provided values.

¹Handles in LightweightVK were inspired by the talk "HypeHype Mobile Rendering Architecture" [Aaltonen 2023].

Table 1: "Bindless" rendering on Android [Kosarevsky and Kuznetsov 2024].

GPU	Resolution	MSAA	Textures size	Frame time
Immortalis-G715	2712x1220	8x	2048x2048	26ms
Immortalis-G715	2712x1220	None	2048x2048	16ms
Immortalis-G715	2712x1220	None	256x256	14ms
Mali-G710	3120x1440	8x	2048x2048	85ms
Mali-G710	3120x1440	None	2048x2048	62ms
Mali-G710	3120x1440	None	256x256	57ms
Mali-G710	2712x1220	8x	2048x2048	80ms
Mali-G710	2712x1220	None	256x256	54ms

4 DISCUSSION

LightweightVK supports Android, and there're performance metrics of its benchmark demo running on the ARM Immortalis-G715 GPU and ARM Mali-G710 [Kosarevsky and Kuznetsov 2024], both featuring ARM's Valhall architecture. Testing was conducted on the Bistro scene shown in the Figure 1, comprising approximately 3 million primitives, uncompressed textures, and generated mipmaps. The results are in the Table 1.

According to their paper [Kosarevsky and Kuznetsov 2024], the best average frame time achieved on the Immortalis-G715 was 14 milliseconds, compared to 54 milliseconds on the Mali-G710. The Immortalis-G715 reduces the average frame time by 4.5 times. Although 14 milliseconds is still not ideal for AR/VR experiences, it can be sufficient for many mobile games. Additional improvements could be achieved by using native texture compression formats (like ASTC) and reducing geometry complexity.

Our exploration of "bindless" rendering in Vulkan has revealed its efficiency on modern mobile hardware and elegance in designing rendering engines across various platforms. Transitioning from the old "bindful" approach to the new "bindless" one is straightforward and incremental, without significant compromises. LightweightVK exemplifies this transition and is freely available under the MIT license <https://github.com/corporateshark/lightweightvk>.

REFERENCES

- Sebastian Aaltonen. 2023. *HypeHype Mobile Rendering Architecture*. <https://advances.realtimerendering.com/s2023/index.html#HypeHype> SIGGRAPH 2023 Conference.
- ARM. 2023. *Immortalis-G715*. <https://developer.arm.com/Processors/Immortalis-G715> Immortalis-G715 Technical Specifications.
- Sergey Kosarevsky. 2024. *LightweightVK*. <https://github.com/corporateshark/lightweightvk> LightweightVK GitHub repository.
- Sergey Kosarevsky and Roman Kuznetsov. 2024. Designing Rendering Engines with "Bindless" Vulkan. In *GPU Zen 3*, Wolfgang Engel (Ed.). Independently Published.
- Sergey Kosarevsky and Viktor Latypov. 2021. *3D Graphics Rendering Cookbook*. Packt Publishing.
- Sergey Kosarevsky and Alexey Medvedev. 2024. *Writing an Efficient Vulkan Renderer for Quest Pro and Quest 3*. <https://vulkan.org/user/pages/09.events/vulkanised-2024/vulkanised-2024-sergey-kosarevsky-alexey-medvedev-meta.pdf> Vulkanised 2024 Conference.
- Roman Kuznetsov. 2024. *MoltenVK for Advanced Vulkan Renderers on macOS*. <https://vulkan.org/user/pages/09.events/vulkanised-2024/vulkanised-2024-roman-kuznetsov-meta.pdf> Vulkanised 2024 Conference.
- Iago Calvo Lista. 2024. *Realistic Graphics with Ray Tracing on Mobile*. <https://vulkan.org/user/pages/09.events/vulkanised-2024/vulkanised-2024-iago-calvo-lista-arm-2.pdf> Vulkanised 2024 Conference.